Übungsblatt 1: Zahlendarstellung, Logik und Register

Mikrocomputertechnik 1

Musterlösung – Rechenwege und begründete Zustandsvorhersagen.

Ziel und Voraussetzungen

Bearbeiten Sie die Aufgaben zuerst auf Papier. Es werden nur die Inhalte von Vorlesung 1 vorausgesetzt. Eine Programmiersprache oder Simulatorinstallation ist für die sechs Papieraufgaben nicht erforderlich.

Geben Sie Bitbreite, Interpretation und Rechenweg an. Unterscheiden Sie Adresse und Inhalt, Übertrag und vorzeichenbehafteten Überlauf sowie kombinatorisches Ergebnis und gespeicherten Zustand.

Aufgabe	Thema	Übungspunkte
1	Binär und Hexadezimal	10
2	Zweierkomplement und Überlauf	12
3	Byteordnung und Adressen	10
4	Logik und Multiplexer	10
5	Volladdierer und Carrykette	8
6	Flanke, Freigabe und Register	10
Summe		**60**

Die Punkte dienen der Selbstkontrolle. Die Aufgaben sind zur Vorbereitung auf ähnliche Klausuraufgaben geeignet; sie legen keine konkreten Prüfungsregeln fest.

Gemeinsames Lehrmodell

Ein Byte hat acht Bit, ein RISC-V-Word 32 Bit. Kleine vier- und achtbittige Beispiele verwenden die jeweils angegebene Breite. Vorzeichenbehaftete Werte werden als Zweierkomplement interpretiert. Little Endian ist die Kursfestlegung, nicht eine Aussage über jede mögliche Rechnerumgebung.

Das Registermodell übernimmt Daten an der **steigenden Flanke** bei Schreibfreigabe `WE=1`. Bei `WE=0` oder ohne aktive Flanke hält es. Setup und Hold, die Stabilitätszeiten vor und nach der Flanke, sind in den Aufgaben erfüllt. Gespeicherte Ausgänge werden nach der Clock-to-Q-Verzögerung betrachtet. Jeder Anfangszustand ist angegeben.

Aufgabe 1: Binär und Hexadezimal (10 Punkte)

Arbeitsauftrag

1. Stellen Sie 85 als **achtbittiges** Binärmuster und als Hexwert dar. Zeigen Sie die Zerlegung in Zweierpotenzen. (4 P)
2. Stellen Sie `0xDA3F` als **16-Bit-Muster** dar. Ordnen Sie jede Hexziffer ihrer Vier-Bit-Gruppe zu. (4 P)
3. Wie viele Hexziffern benötigt ein Byte? Wie viele benötigt ein RISC-V-Word? Begründen Sie mit der Breite einer Hexziffer. (2 P)

Führende Nullen gehören zur ausdrücklich geforderten Breite. Das Präfix `0x` kennzeichnet Hexadezimal; Leerzeichen dienen nur der Gruppierung.

Musterlösung

1. $85 = 64 + 16 + 4 + 1$. Das Acht-Bit-Muster ist `0101 0101`. Die Gruppen `0101` und `0101` sind jeweils Hexziffer 5, also `0x55`.
2. D ist dreizehn, A zehn, 3 drei und F fünfzehn:

Hexziffer	D	A	3	F
Vierergruppe	`1101`	`1010`	`0011`	`1111`

Ergebnis: `1101 1010 0011 1111`, insgesamt 16 Bit. Die Gruppen werden nicht umgedreht.

3. Eine Hexziffer bildet vier Bit ab. Ein Byte benötigt $8/4 = 2$ Hexziffern, ein RISC-V-Word $32/4 = 8$.

Aufgabe 2: Zweierkomplement und Überlauf (12 Punkte)

Arbeitsauftrag

1. Bilden Sie die Darstellung von -85 mit acht Bit. Zeigen Sie positives Ausgangsmuster, invertiertes Muster und Addition eins. (4 P)
2. Lesen Sie `0xAB` vorzeichenlos und als Acht-Bit-Zweierkomplement. Welche Vorzeichenerweiterung auf 32 Bit erhält den negativen Wert? (3 P)
3. Ergänzen Sie die Tabelle für **acht gespeicherte Bits**. Prüfen Sie Übertrag und Signed-Überlauf getrennt. (3 P)
4. Negieren Sie `1000 0000` modular durch Invertieren und Addition eins. Warum stellt das Ergebnis nicht plus 128 dar? (2 P)

Musterlösung

1. Plus 85: `0101 0101` beziehungsweise `0x55`. Invertiert: `1010 1010`, `0xAA`. Plus eins: `1010 1011`, `0xAB`.
2. Vorzeichenlos: $128+32+8+2+1=171$. Als Zweierkomplement: $-128+32+8+2+1=-85$. Vorzeichenerweiterung: `0xFFFFFFAB`, da das höchste Ausgangsbit eins ist.

Rechnung	Ergebnisbits	Übertrag	Signed-Überlauf
`0x7F + 1`	`1000 0000`	0	Ja: 128 passt nicht
`0xFF + 1`	`0000 0000`	1	Nein: $-1+1=0$

3. Der Übertrag ist das neunte Bit. Signed-Überlauf betrifft dagegen den mathematischen Bereich von −128 bis 127.
4. `1000 0000` → `0111 1111` → `1000 0000`. Das letzte Muster bedeutet im Acht-Bit-Zweierkomplement weiterhin −128. Plus 128 ist nicht darstellbar. Zweimalige modulare Negation liefert trotzdem auch für diesen Fall das ursprüngliche Muster.

Aufgabe 3: Byteordnung und Adressen (10 Punkte)

Arbeitsauftrag

Das 32-Bit-Wort `0xAABBCCDD` beginnt im angenommenen byteadressierten Speicher an `0x1000`.

1. Tragen Sie die Bytes für Little Endian und Big Endian ein. Welche Adresse enthält bei Little Endian das höchstwertige Byte? (5 P)
2. Ist die Startadresse `0x1000` für ein Vier-Byte-Word natürlich ausgerichtet? Gilt das auch für `0x1001`? Begründen Sie. (2 P)
3. Erklären Sie, weshalb Little Endian keine Umkehrung der Bits innerhalb eines Bytes bedeutet. (1 P)
4. Ein Modell hat 32-Bit-Byteadressen. Berechnen Sie die Größe seines Adressraums in GiB. Warum folgt daraus nicht dieselbe Menge eingebauter Arbeitsspeicher? (2 P)

Musterlösung

Adresse	Little Endian	Big Endian
`0x1000`	`DD`	`AA`
`0x1001`	`CC`	`BB`
`0x1002`	`BB`	`CC`
`0x1003`	`AA`	`DD`

1. Das höchstwertige Byte AA liegt bei Little Endian an `0x1003`.
2. `0x1000` ist durch vier teilbar, `0x1001` nicht. Die Ausrichtung betrifft die Adresse und ist unabhängig von der Byteordnung.
3. Die Bytes DD, CC, BB und AA bleiben jeweils unverändert. Geändert wird ihre Zuordnung zu aufsteigenden Adressen, nicht ihre innere Bitfolge.
4. 2^{32} mögliche Byteadressen ergeben $2^{32}/2^{30} = 4$ GiB. Bereiche können fehlen, unzugänglich sein oder anderen Funktionen dienen. Adressierbarkeit ist keine Angabe der installierten Speichermenge.

Aufgabe 4: Logik und Multiplexer (10 Punkte)

Arbeitsauftrag

Ein 2-zu-1-Multiplexer hat Dateneingänge A, B, Auswahl S und Ausgang Y. Bei $S = 0$ wählt er A, bei $S = 1$ wählt er B.

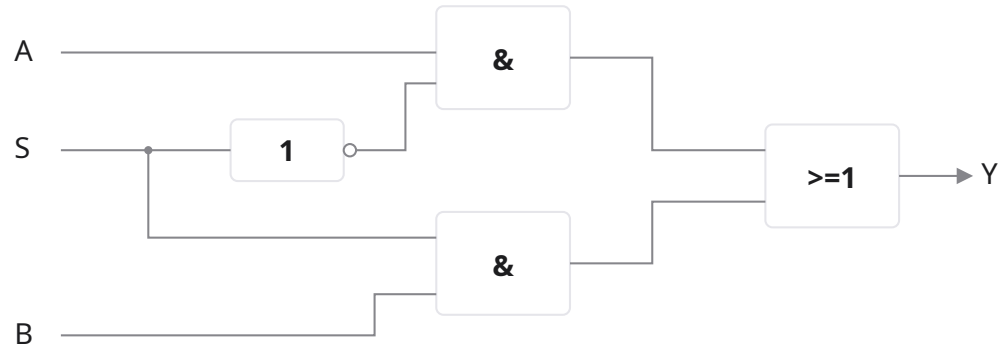
1. Vervollständigen Sie alle acht Tabellenzeilen. (4 P)
2. Geben Sie Y mit UND, ODER und Negation an und skizzieren Sie eine passende Gatterschaltung. Beschriften Sie Eingänge, Ausgang und Verzweigungen. (4 P)
3. Wortauswahl mit 32 Bit: $A = 5$, $B = 10$, $S = 1$. Welches Wort entsteht und wie viele Ein-Bit-Multiplexer benötigt unser Parallelmodell? (2 P)

Musterlösung

Die Ausgangsspalte lautet in Reihenfolge: **0, 0, 1, 1, 0, 1, 0, 1**.

$$Y = (\overline{S} \wedge A) \vee (S \wedge B)$$

S wird negiert und mit A verundet. S wird direkt mit B verundet. Ein ODER verbindet die beiden Teilergebnisse.



Die Wortauswahl liefert 10. Unser Parallelmodell benötigt 32 Ein-Bit-Multiplexer mit demselben Auswahlwert, nicht 32 Ausführungstakte.

Aufgabe 5: Volladdierer und Carrykette (8 Punkte)

Arbeitsauftrag

1. Bestimmen Sie Summenbit S und ausgehenden Übertrag C_{out} für $(A, B, C_{\text{in}}) = (1, 1, 0)$ und $(1, 1, 1)$. Begründen Sie durch die mathematische Summe. (2 P)
2. Addieren Sie **vorzeichenlos** `0110` und `0101` mit vier gespeicherten Bits und Startübertrag null. Ergänzen Sie je Stelle die Eingangsüberträge, Summenbits und Ausgangsüberträge. Nennen Sie Endwort und finalen Übertrag. (3 P)
3. Berechnen Sie $7 - 3$ mit vier gespeicherten Bits als Addition mit invertierten B-Bits und Startübertrag. Zeigen Sie, an welcher Stelle die zusätzliche Eins eingespeist wird. (3 P)

Musterlösung

1. Zwei Einsen ergeben 2, also $S = 0$, $C_{\text{out}} = 1$. Drei Einsen ergeben 3, also $S = 1$, $C_{\text{out}} = 1$. Stets gilt $A + B + C_{\text{in}} = S + 2C_{\text{out}}$.

Stelle von rechts	A_i	B_i	C_i	S_i	C_{i+1}
0	0	1	0	1	0
1	1	0	0	1	0
2	1	1	0	0	1
3	0	0	1	1	0

2. Das Wort wird von höchster zu niedrigster Stelle geschrieben: `1011`, vorzeichenlos 11. Finaler Übertrag: 0.
3. A ist `0111`, B `0011`, invertiertes B `1100`. Nur der erste Übertrag `C0=1` wird vorgegeben. Höhere Stellen übernehmen den Übertrag der vorherigen Stelle. Die Addition ergibt `10100`, gespeichert werden die unteren vier Bit `0100`, also 4. Der äußere Übertrag ist 1.

Aufgabe 6: Flanke, Freigabe und Register (10 Punkte)

Arbeitsauftrag

1. Anfang: das Wortregister hält $Q = 3$. Ergänzen Sie Q nach jeder Zeile. Die Schritte sind nacheinander auszuführen, alle Zeitbedingungen sind erfüllt. (5 P)
2. Warum kann Q bei demselben aktuellen Eingang D verschiedene Werte haben? Ist jede sequenzielle Schaltung zwingend flankengesteuert? Nennen Sie ein Gegenbeispiel aus der Vorlesung. (3 P)
3. Anfang `x5=7`, `x6=3`, `x7=0`. Was steht nach `add x7, x5, x6` und gültiger Registerübernahme in diesen Registern? Was wäre bei Ziel `x0` dessen gespeicherter Wert? (2 P)

Musterlösung

Ereignis	D	WE	Q danach
Keine aktive Flanke	5	1	3
Steigende Flanke	5	1	5
Keine aktive Flanke	7	1	5
Steigende Flanke	7	0	5
Steigende Flanke	9	1	9

1. Schreibfreigabe allein übernimmt keine Daten. Erst eine aktive Flanke mit Freigabe ersetzt den gespeicherten Wert.
2. Der bisher gespeicherte Zustand hängt von vorherigen Übernahmen ab. Sequenziell bedeutet Zustand, nicht allgemein Taktflanke. Ein Latch ist während seines Freigabepegels durchlässig und hält sonst.
3. `x5=7` und `x6=3` bleiben erhalten, `x7=10`. Ein Schreibziel `x0` speichert die Summe nicht: `x0` bleibt null.

Optional: Registeraddition mit Ripes beobachten

Nutzen Sie diesen Versuch bei vorbereiteter Ripes-Installation. Für die Papieraufgaben ist er nicht erforderlich.

1. Notieren Sie Ripes-Version und gewähltes RV32I-Prozessormodell. Ein Single-Cycle-Modell ist für den ersten Versuch übersichtlich.
2. Öffnen Sie `beispiele/vl1_registeraddition.s`. Keine Ein-/Ausgabegeräte oder Systemdienste sind nötig.
3. Starten Sie neu und führen Sie die Anweisungen schrittweise aus. Halten Sie nach der dritten Rechenanweisung an und notieren Sie `x0`, `x5`, `x6` und `x7`.
4. Verfolgen Sie Lesen, Rechnen und die tatsächliche Registerübernahme im gewählten Modell. Unterscheiden Sie die Registernummer von dem gezeigten Datenwert.
5. Erklären Sie, weshalb die anschließende Schleife die Werte erhält und keine Konsolenausgabe benötigt.

`addi` addiert eine direkt angegebene Konstante zu einem Registerwert. `add` addiert zwei Registerwerte. Das Ziel steht zuerst. Die Marke `main` benennt den Einstieg, `j inspect` springt immer zur Marke `inspect` zurück. Die Datei ist vollständig und bleibt zur Zustandsinspektion in dieser Schleife.

Sollverhalten und Auswertung

Die beiden `addi`-Anweisungen bilden null plus sieben beziehungsweise null plus drei. `add x7, x5, x6` schreibt zehn ins Zielregister. Erwartet sind `x0=0`, `x5=7`, `x6=3`, `x7=10`.

Die Dauerschleife führt einen Sprung aus und verändert diese Datenregister nicht. Wann ein Zwischenwert im Datenpfad und wann der neue Registerwert sichtbar wird, ist anhand des gewählten Modells zu protokollieren. Ein Simulatorschritt ist keine gemessene Hardwarezeit. Eigene Beobachtungen stammen aus dem eigenen Lauf; die Sollwerte ersetzen kein solches Protokoll.